

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и ОА в ИЗ»
на тему «Динамические списки»

Выполнили:
студенты группы 25ВВВ1
Новиков И.А.
Евстратов В.С.

Принял:
к.т.н., доцент Юрова О.В.

Пенза 2026

Цель работы

Изучение динамических структур данных языка Си на примере односвязных списков, освоение работы с указателями и динамической памятью (malloc, free), а также реализация на основе односвязного списка трёх структур данных: приоритетной очереди, очереди (FIFO) и стека (LIFO).

Лабораторное задание

На основе приведённого в методических указаниях кода односвязного списка выполнить следующие задания:

Задание 1: Реализовать приоритетную очередь путём добавления элемента в список в соответствии с приоритетом объекта (объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Задание 2: Реализовать структуру данных «Очередь» (принцип FIFO – первым пришёл, первым вышел).

Задание 3: Реализовать структуру данных «Стек» (принцип LIFO – последним пришёл, первым вышел).

Описание метода решения задач

Во всех заданиях используется один и тот же односвязный линейный список. Элемент списка описывается структурой, в которую, по сравнению с исходным кодом, добавлено целочисленное поле приоритета priority (значимо только для приоритетной очереди):

```
struct node {  
    char inf[256];  
    int priority;  
    struct node* next;  
};
```

Обращение к списку осуществляется через два глобальных указателя: head – на первый элемент и last – на последний. Каждое задание перед началом работы обнуляет оба указателя, поэтому структуры данных не влияют друг на друга. Память под элемент выделяется функцией malloc (в функции get_struct), освобождается функцией free. Из исходного кода сохранены функции просмотра (review), поиска (find) и удаления по содержимому (del); добавлены функция освобождения всего списка free_list и функция просмотра с выводом приоритета review_pq.

Задание 1 (приоритетная очередь, функция `pq_add`): список поддерживается упорядоченным по убыванию приоритета. После ввода имени и приоритета возможны два случая. Если список пуст или приоритет нового элемента строго больше приоритета головы, элемент становится новой головой списка. Иначе выполняется проход по списку двумя указателями (`prev` – предыдущий, `cur` – текущий) пока приоритет текущего элемента больше или равен приоритету нового; затем элемент вставляется между `prev` и `cur` (если `cur == NULL`, то в конец, и указатель `last` обновляется). Условие «больше или равен» гарантирует, что элементы с одинаковым приоритетом остаются в порядке поступления. Первым в списке всегда стоит объект с наибольшим приоритетом.

Задание 2 (очередь, функции `queue_add` и `queue_remove`): добавление элемента выполняется в конец списка через указатель `last` за время $O(1)$: если список пуст, `head` и `last` указывают на новый элемент, иначе `last->next = p` и `last = p`. Извлечение выполняется из начала: голова переносится на следующий элемент, прежний первый элемент освобождается; если очередь стала пустой, `last` обнуляется. Таким образом, первым извлекается элемент, добавленный раньше всех.

Задание 3 (стек, функции `stack_push` и `stack_pop`): добавление (`push`) выполняется в начало списка: `p->next = head`, затем `head = p`. Извлечение (`pop`) снимает элемент с вершины, то есть из головы списка, и освобождает память. Обе операции выполняются за время $O(1)$, первым извлекается элемент, добавленный последним.

Сводка операций: в приоритетной очереди элемент добавляется в позицию, соответствующую его приоритету (сложность добавления $O(n)$), а извлекается из начала списка; в очереди (FIFO) элемент добавляется в конец списка (сложность $O(1)$) и извлекается из начала; в стеке (LIFO) элемент добавляется в начало списка (сложность $O(1)$) и извлекается также из начала.

Пояснительный текст к программе

Структура программы: при запуске выводится меню выбора задания (1 – приоритетная очередь, 2 – очередь, 3 – стек), после чего вызывается соответствующая функция `task1`, `task2` или `task3`. Каждая из них содержит цикл с собственным меню операций; при выборе пункта «Exit» список полностью освобождается функцией `free_list`.

Основные функции программы: `get_struct` создаёт элемент списка (выделяет память с проверкой и вводит имя); `review` и `review_pq` выводят список от головы к концу, вторая – вместе с приоритетом; `find` ищет элемент по содержимому и возвращает указатель на найденный элемент; `del` удаляет элементы с заданным содержимым с корректировкой указателей; `free_list` освобождает память всех элементов списка; `pq_add` вставляет элемент в приоритетную очередь в позицию, соответствующую приоритету; `queue_add` и `queue_remove` добавляют элемент в конец и извлекают из начала очереди; `stack_push` и `stack_pop` добавляют элемент в вершину и снимают с вершины стека; `task1`, `task2` и `task3` содержат меню и основной цикл работы с приоритетной очередью, очередью и стеком.

Динамическое управление памятью: элементы создаются функцией `malloc` с проверкой результата на `NULL` и освобождаются функцией `free` при удалении или при выходе из режима работы. Это исключает утечки памяти.

Обработка граничных случаев: при работе с пустым списком функции просмотра, поиска и удаления выводят сообщение «List is empty», при извлечении из пустой очереди или стека – «Queue is empty» или «Stack is empty», при отсутствии искомого элемента – «Not found». При удалении последнего элемента и извлечении единственного элемента указатель `last` корректируется. Полный текст программы приведён в приложении.

Результаты работы программы

Приоритетная очередь. Добавлены объекты TaskA (приоритет 2), TaskB (5), TaskC (2) и TaskD (5). При просмотре объекты расположены по убыванию приоритета, а объекты с равным приоритетом – в порядке добавления. Затем выполнены поиск объекта TaskC и удаление объекта TaskA.

```
lab3
1 - priority queue
2 - queue (FIFO)
3 - stack (LIFO)
> 1
priority queue

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 1
Enter name: TaskA
Enter priority: 2

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 1
Enter name: TaskB
Enter priority: 5

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 1
Enter name: TaskC
Enter priority: 2

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> TaskD
Invalid input

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 1
Enter name: TaskD
Enter priority: 5

1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 2
TaskB (priority 5)
TaskD (priority 5)
TaskA (priority 2)
TaskC (priority 2)
```

```
1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 3
Enter name: TaskC
Found: TaskC (priority 2)
```

```
1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 4
Enter name: TaskA
```

```
1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
> 2
TaskB (priority 5)
TaskD (priority 5)
TaskC (priority 2)
```

```
1 - Add
2 - View
3 - Find
4 - Delete
5 - Exit
>
```

Очередь. Добавлены объекты Alpha, Beta и Gamma; извлечение происходит в порядке добавления. После этого проверены поиск существующего элемента и удаление отсутствующего элемента (Beta уже извлечён).

```
lab3
1 - priority queue
2 - queue (FIFO)
3 - stack (LIFO)
> 2
queue (FIFO)

1 - Enqueue
2 - Dequeue
3 - View
4 - Find
5 - Delete
6 - Exit
> 1
Enter name: Alpha

> 1
Enter name: Beta

> 1
Enter name: Gamma

> 3
Alpha
Beta
Gamma

> 2
Removed: Alpha

> 2
Removed: Beta

> 4
Enter name: Gamma
Found: Gamma

> 5
Enter name: Beta
Not found

> 3
Gamma

> 6
```

Стек. Добавлены объекты One, Two и Three; извлечение происходит в обратном порядке – первым снимается Three. Попытка извлечения из пустого стека приводит к сообщению об ошибке.

```
lab3
1 - priority queue
2 - queue (FIFO)
3 - stack (LIFO)
> 3
stack (LIFO)

1 - Push
2 - Pop
3 - View
4 - Find
5 - Delete
6 - Exit
> 1
Enter name: One

> 1
Enter name: Two

> 1
Enter name: Three

> 3
  Three
  Two
  One

> 2
Popped: Three

> 3
  Two
  One

> 2
Popped: Two

> 2
Popped: One

> 2
Stack is empty

> 6
```

Выводы

В ходе выполнения лабораторной работы №3 были изучены:

1. Принципы организации односвязных динамических списков и работа с их элементами через указатели.
2. Функции динамического выделения и освобождения памяти (malloc, free) при создании и удалении элементов списка.
3. Алгоритм вставки элемента в упорядоченный список, позволяющий реализовать приоритетную очередь с сохранением порядка поступления элементов с равным приоритетом.
4. Реализация структур данных «Очередь» (FIFO) и «Стек» (LIFO) на основе односвязного списка: очередь требует указателя на конец списка для добавления за $O(1)$, стек работает только с головой списка.
5. Обработка граничных случаев (пустой список, удаление единственного или последнего элемента) и освобождение памяти при завершении работы.